

DepGuard AI: The Autonomous Dependency Architect

Đề xuất Dự án Hackathon — Hệ thống Agentic AI

Mục lục

- Tổng quan Dự án
- Vấn đề & Giải pháp
- Kiến trúc Hệ thống Agentic AI
 - 3.1 Goal-Driven — Định hướng Mục tiêu
 - 3.2 Planning — Lập kế hoạch Tự động
 - 3.3 Autonomous — Tự trị & Chủ động
 - 3.4 Feedback Loop — Vòng lặp Phản hồi
- Các Tính năng Cốt lõi
- Kiến trúc Kỹ thuật
- Điểm Khác biệt & Sáng tạo
- Luồng Hoạt động Demo
- Tech Stack Đề xuất

1. Tổng quan Dự án

Hạng mục Chi tiết

Tên dự án DepGuard AI: The Autonomous Dependency Architect

Lĩnh vực DevTools × Agentic AI

Đối tượng Developer cá nhân, nhóm kỹ thuật, DevOps Engineer, Tech Lead

Mô hình Multi-Agent System với Static Analysis + LLM Code Refactoring + Registry Polling

Điểm cốt lõi Không chỉ cảnh báo — Agent tự đọc code, tự hiểu tác động, tự viết lại đoạn code cần thay đổi chỉ bằng một nút bấm

Tầm nhìn: Developer không còn phải dành hàng giờ đọc changelog, kiểm tra breaking changes hay sửa từng file khi nâng cấp dependency. DepGuard AI là người đồng đội kỹ thuật tự trị — luôn thức, luôn theo dõi, luôn sẵn sàng hành động.

2. Vấn đề & Giải pháp

2.1 Vấn đề Thực tế — "The Dependency Hell"

Mọi dự án phần mềm hiện đại đều sống nhờ các thư viện bên ngoài. Và đây là cơn ác mộng quen thuộc:

```
my-project/
├─ package.json      ← 47 dependencies, nhiều cái cũ 2 năm
├─ requirements.txt   ← Django 3.2 (EOL), numpy 1.21 (lỗi bảo mật!)
├─ pom.xml            ← Spring Boot 2.7 (sắp EOL)
├─ go.mod             ← 12 module, 5 cái có breaking changes mới
└─ src/
   └─ api/handler.py  ← Dùng API đã bị deprecated từ numpy 1.24
   └─ components/App.jsx ← React 17 syntax, không tương thích React 19
   └─ utils/db.java    ← Hibernate query cũ, không chạy được v6
```

Bốn "nỗi đau" cốt lõi của developer:

Vấn đề	Hệ quả thực tế
Không biết version nào đã lỗi thời	Dự án chạy được hôm nay, sụp đổ lạng lề vì security patch bị bỏ qua
Không dám update	"Nếu tôi update React lên 19, không biết cái gì vỡ" → Nợ kỹ thuật tích lũy
Breaking changes ần	Update xong → build fail → mất 2 ngày debug tìm nguyên nhân
Đa ngôn ngữ / đa file config	<code>package.json</code> , <code>requirements.txt</code> , <code>pom.xml</code> , <code>go.mod</code> — mỗi cái một tool riêng

2.2 Các Giải pháp Hiện có và Giới hạn

Dependabot / Renovate Bot:

- ✓ Tạo Pull Request tự động khi có version mới
- ✗ Không hiểu code của bạn – chỉ bump version số, không sửa code
- ✗ Không giải thích tác động cụ thể lên project của bạn
- ✗ Không phân biệt "safe update" vs "breaking change cần sửa 20 file"

npm audit / pip-audit / OWASP Dependency Check:

- ✓ Phát hiện lỗ hổng bảo mật
- ✗ Chỉ báo lỗi, không hành động
- ✗ Không cung cấp migration plan

IDE Warnings (IntelliJ, VS Code):

- ✓ Highlight API deprecated ngay trong editor
- ✗ Không tự động fix
- ✗ Không có cái nhìn tổng thể toàn project

2.3 Giải pháp: DepGuard AI

DepGuard AI là **Agentic AI System** đầu tiên kết hợp cả ba năng lực:

PHÁT HIỆN

Quét toàn bộ dependency files Theo dõi registry

PHÂN TÍCH

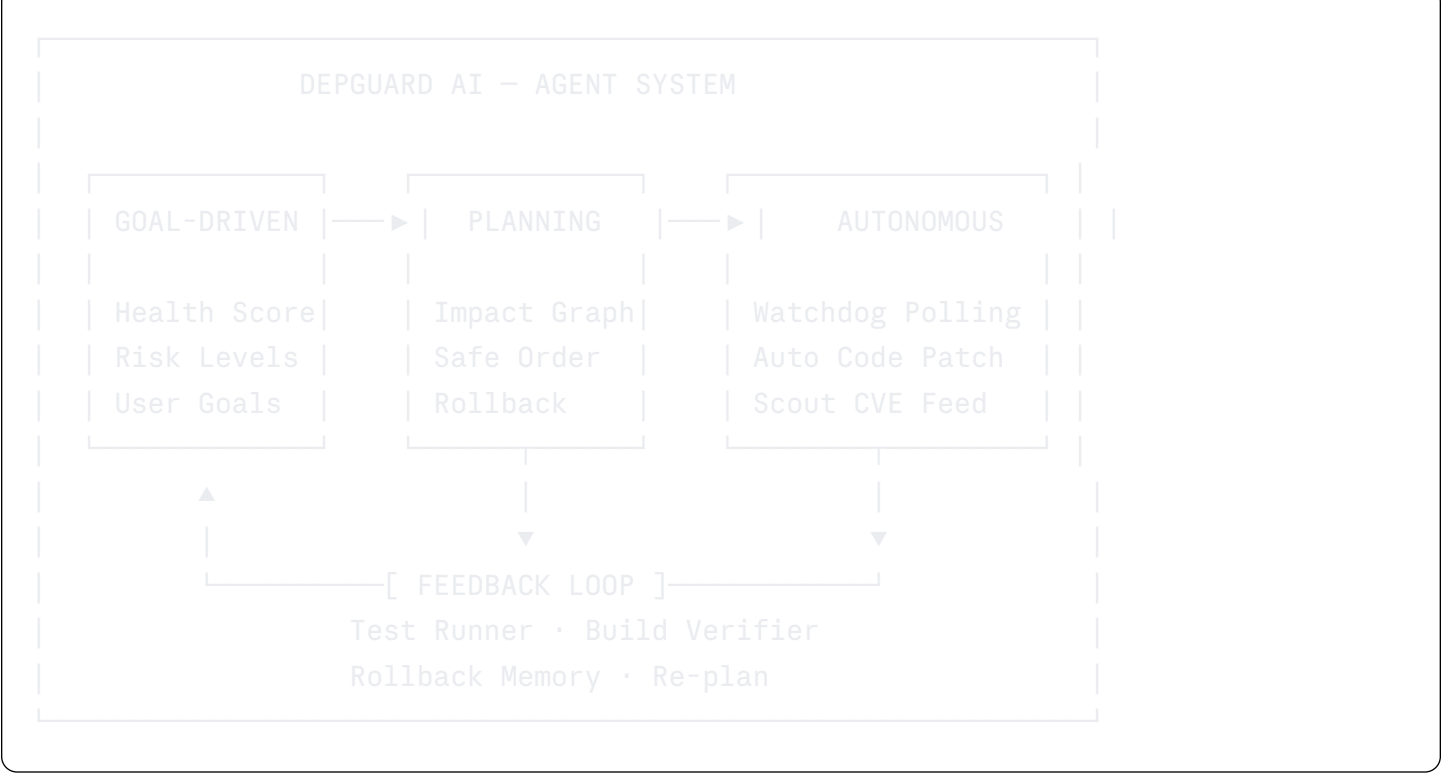
Đọc code, hiểu tác động thực Breaking change
--

HÀNH ĐỘNG

Viết lại code, cập nhật version Chỉ bằng 1 nút bấm
--

3. Kiến trúc Hệ thống Agentic AI

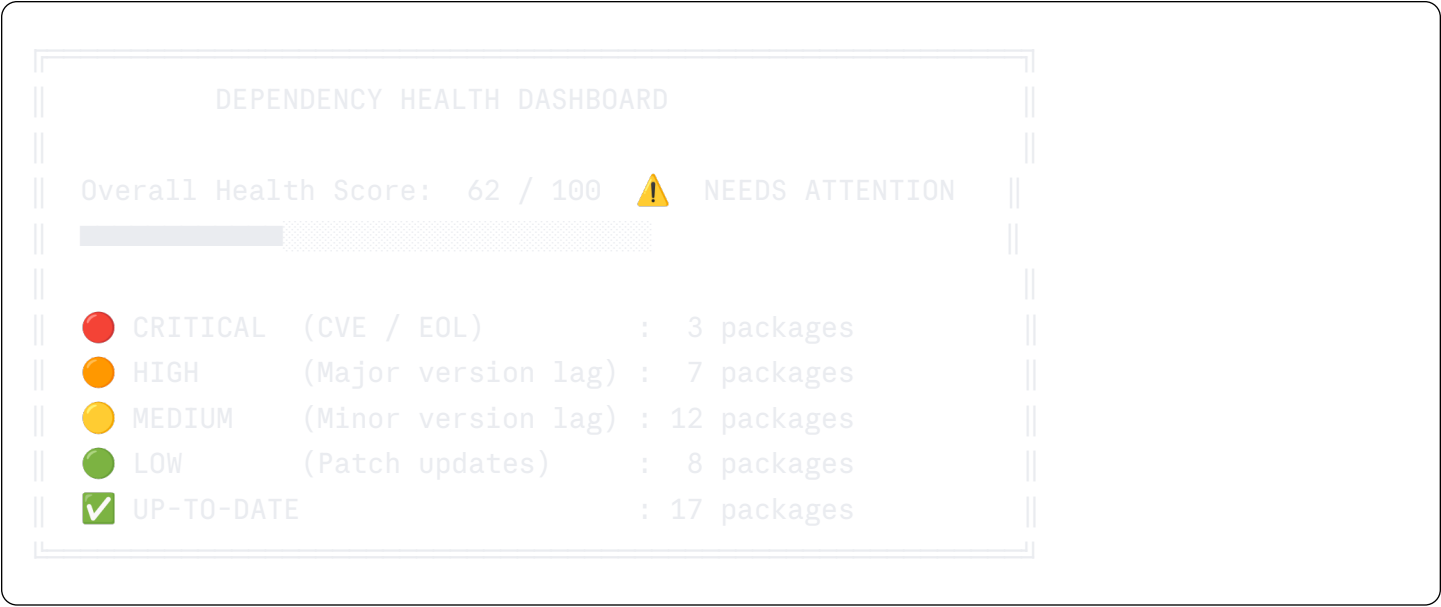
DepGuard AI được thiết kế trên **4 trụ cột** của một Agentic AI System hoàn chỉnh:



3.1 Goal-Driven — Định hướng Mục tiêu

Mục tiêu tối thượng: Đưa project về trạng thái "**Dependency Health = 100%**" — không có lỗ hổng bảo mật, không có version EOL, không có deprecated API nào còn tồn tại trong code.

Hệ thống đo lường mục tiêu — Dependency Health Score:



Ba cấp độ mục tiêu người dùng có thể cấu hình:

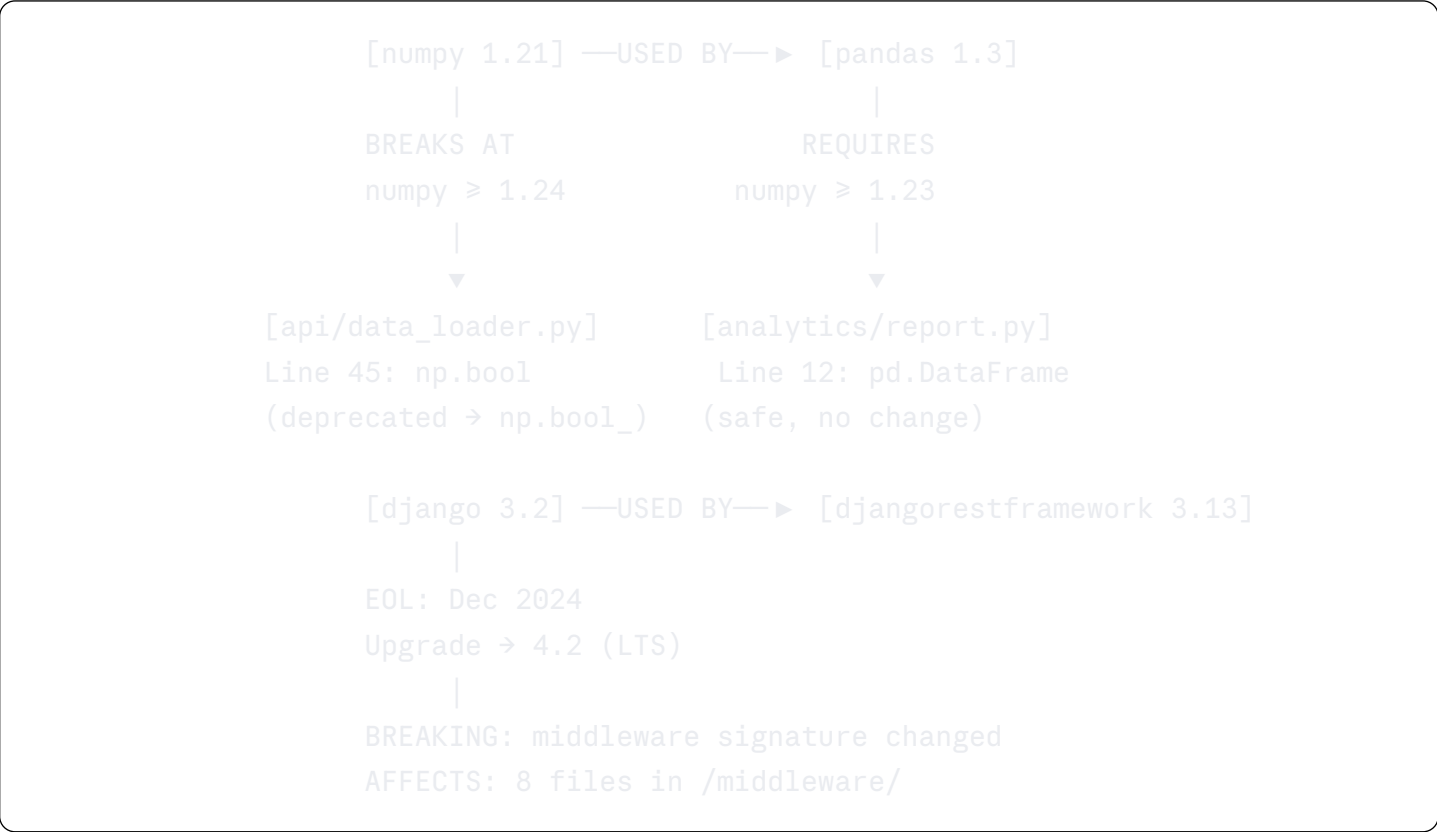
Chế độ	Mục tiêu	Hành vi Agent
 Security-First	Chỉ vá lỗ hổng CVE, không đụng feature updates	Chỉ patch CRITICAL + HIGH CVE
 Balanced	Cân bằng giữa an toàn và tính năng mới	Patch CVE + update Minor versions
 Always Latest	Luôn theo kịp version mới nhất	Đề xuất tất cả, bao gồm Major updates

Agent luôn nhớ mục tiêu này và **từ chối** tự động làm các thay đổi vượt quá phạm vi người dùng cho phép.

3.2 Planning — Lập kế hoạch Tự động

Thách thức lớn nhất khi update dependency không phải là "update cái gì" mà là "**update theo thứ tự nào để không vỡ hệ thống**".

Bước 1: Xây dựng Dependency Impact Graph



Bước 2: Tính toán Safe Update Order

Agent phân tích đồ thị phụ thuộc và xác định thứ tự update an toàn:

UNSAFE ORDER (sẽ gây conflict):

- × pandas 1.3 → 2.0 (cần numpy ≥ 1.23)
- × numpy 1.21 → 1.26 (breaking change np.bool)
- Build FAIL vì numpy update xong mà code chưa sửa

SAFE ORDER (do Agent lập):

- Step 1: Sửa code trước (np.bool → np.bool_ ở 3 file)
- Step 2: Update numpy 1.21 → 1.26 (patch tương thích)
- Step 3: Update pandas 1.3 → 2.0 (đã có numpy mới)
- Step 4: Chạy test suite xác nhận
- Build SUCCESS 

Bước 3: Tạo Migration Plan chi tiết

```
|| MIGRATION PLAN - numpy 1.21 → 1.26 ||
||
||  Impact Assessment: ||
||   • Files affected      : 5 files ||
||   • Lines to change     : 12 lines ||
||   • Risk level          : MEDIUM (API rename, not removal) ||
||   • Estimated time      : ~3 minutes (auto) / ~2 hours (manual)||
||
||  Code Changes Required: ||
|| 1. api/data_loader.py:45 - np.bool → np.bool_ ||
|| 2. api/data_loader.py:89 - np.int  → np.int_ ||
|| 3. utils/matrix.py:23    - np.complex → np.complex_ ||
|| 4. utils/matrix.py:67    - np.object → np.object_ ||
|| 5. tests/test_data.py:12 - np.bool → np.bool_ ||
||
||  Prerequisite: None ||
||  Must complete before: pandas update ||
||
|| [  Auto-Fix & Update ] [  Review First ] [  Skip ] ||
```

Bước 4: Rollback Strategy Planning

Trước khi thực hiện bất kỳ thay đổi nào, Agent lập sẵn kế hoạch rollback:

```
python
```

```
# Agent tự tạo rollback checkpoint trước mỗi update
rollback_plan = {
    "checkpoint_id": "ckpt_20250601_143022",
    "git_stash": "stash@{0}",          # Code changes stash
    "lockfile_backup": "package.lock.bak", # Exact versions snapshot
    "rollback_command": "depguard rollback ckpt_20250601_143022",
    "auto_rollback_trigger": "build_fail OR test_fail"
}
```

3.3 Autonomous — Tự trị & Chủ động

Đây là trụ cột tạo sự khác biệt hoàn toàn với Dependabot hay Renovate:

Cơ chế 1: Watchdog — Giám sát liên tục, không cần kích hoạt thủ công

```
WATCHDOG AGENT
(Chạy ngầm 24/7)

Mỗi 6 giờ: Poll PyPI / npm / Maven Central / pkg.go.dev
Tức thì: Theo dõi CVE feeds (NVD, OSV, GitHub Advisory)
Mỗi commit: Quét lại dependency files trong repo

Phát hiện → Phân loại → Ưu tiên → Thông báo

● CVE Critical → Push notification NGAY LẬP TỨC
● Major update → Báo cáo hàng ngày
● Minor update → Báo cáo hàng tuần
```

Ví dụ thông báo minh:

 [CRITICAL – Hành động ngay] 14:32, 01/06/2025

CVE-2025-XXXX phát hiện trong requests==2.28.0

- Mức độ: CRITICAL (CVSS 9.8)
- Loại: Remote Code Execution
- Fix: Nâng lên requests==2.32.0
- Ảnh hưởng đến project của bạn:
 - src/api/client.py (line 8, 45, 67)
 - tests/test_integration.py (line 12)

[ Tự động vá ngay] [ Xem chi tiết] [ Nhắc sau 1 giờ]

Cơ chế 2: Code Intelligence Agent — Tự đọc và hiểu code

Không chỉ bump version số — Agent thực sự **đọc code** để hiểu tác động:

Phân tích sử dụng API trong codebase:

Quét toàn bộ .py files...

Tìm thấy: np.bool (deprecated tại numpy 1.20, removed tại 1.24)

- └─ api/data_loader.py:45 → np.bool(x)
- └─ api/data_loader.py:89 → dtype=np.bool
- └─ utils/matrix.py:23 → np.bool array

Tìm thấy: pd.DataFrame.append (deprecated tại pandas 1.4, removed 2.0)

- └─ analytics/report.py:102 → df.append(new_row)
- └─ analytics/report.py:118 → result.append(df2)

Tìm thấy: requests.get() — Usage pattern OK 
(API không thay đổi trong version mới)

Kết luận: 5 vị trí cần sửa code TRƯỚC khi update

Cơ chế 3: One-Click Auto-Patch — Sửa code tự động

Khi người dùng nhấn "**Auto-Fix**", Agent thực hiện toàn bộ pipeline:

[Người dùng nhấn: 🤖 Auto-Fix & Update]

↓

[1] Tạo checkpoint (git stash + lockfile backup)

↓

[2] Code Patch Agent viết lại các đoạn code cần sửa:

```
BEFORE (data_loader.py:45):  
    mask = np.bool(condition)
```

```
AFTER (tự động sinh bởi LLM):  
    mask = np.bool_(condition) # Updated: np.bool removed in numpy 1.24
```

↓

[3] Update version trong dependency file:

```
requirements.txt: numpy==1.21.0 → numpy==1.26.4
```

↓

[4] Chạy test suite tự động

```
pytest tests/ ... ✅ 47 passed, 0 failed
```

↓

[5] Báo cáo hoàn thành:

```
"✅ numpy đã được cập nhật thành công.  
5 dòng code đã được sửa tự động.  
Tất cả 47 tests pass."
```

Cơ chế 4: Scout Agent — Chủ động nghiên cứu breaking changes

Khi phát hiện Major version update (e.g., React 17 → 19):

↓

Scout Agent tự động:

1. Đọc official Changelog từ GitHub/npm/PyPI
2. Trích xuất danh sách Breaking Changes
3. Cross-reference với codebase của người dùng
4. Chỉ báo cáo những breaking changes THỰC SỰ ảnh hưởng

↓

Kết quả:

```
"React 19 có 12 breaking changes.  
Project của bạn chỉ bị ảnh hưởng bởi 2:  
1. ReactDOM.render() → createRoot() [4 files]  
2. defaultProps cho function components [1 file]  
Phần còn lại không ảnh hưởng đến codebase của bạn."
```

3.4 Feedback Loop — Vòng lặp Phản hồi

Hệ thống học từ lịch sử update của từng project để ngày càng chính xác hơn.

Kênh 1: Build & Test Verification — Tự xác nhận kết quả

Sau mỗi lần Auto-Fix:

PHASE 1: Static Analysis

✓ Syntax check — không lỗi cú pháp

✓ Import resolution — tất cả module tìm thấy

✓ Type checking (mypy/TypeScript) — không type error

PHASE 2: Test Runner

✓ Unit tests : 47/47 passed

✓ Integration : 12/12 passed

⚠ E2E tests : 1/8 failed → [Chi tiết lỗi]

↓

→ Auto-Rollback kích hoạt nếu test fail vượt ngưỡng

→ Agent ghi nhận: "numpy update gây lỗi E2E test #7"

→ Re-plan: Thêm bước sửa E2E test vào migration plan

Kênh 2: Rollback Memory — Học từ thất bại

Rollback History Database:

2025-05-10	sqlalchemy 1.4 → 2.0	
Kết quả:	ROLLED BACK	
Lý do:	Session.execute() thay đổi return type	
Bài học:	Major SQLAlchemy update cần sửa tất cả raw SQL queries trước khi update	

2025-05-22	axios 0.27 → 1.6	
Kết quả:	SUCCESS ✓	
Ghi chú:	Chỉ cần thay error handling pattern	
Bài học:	axios 1.x error shape thay đổi, pattern fix đơn giản và an toàn	

→ Lần tiếp gặp SQLAlchemy Major update:

Agent tự động thêm cảnh báo: "Lần trước update SQLAlchemy bị rollback vì raw SQL queries. Tôi đã quét và tìm thấy 23 raw queries trong project. Đề xuất: review thủ công trước."

Kênh 3: User Feedback Integration

Người dùng đánh dấu một suggestion là "Không hữu ích":

↓

Agent ghi nhận pattern:

"User project này dùng internal numpy fork –
đừng đề xuất update numpy theo PyPI registry"

↓

Điều chỉnh: Loại numpy khỏi watchlist tự động

Hỏi lại: "Bạn có muốn tôi theo dõi dependency nào khác thay thế?"

Kênh 4: Confidence Score Calibration

Mỗi lần Auto-Fix:

→ Nếu BUILD PASS: Confidence Score của pattern đó +5%

→ Nếu ROLLBACK: Confidence Score -20%, gắn flag "cần review thủ công"

Hiển thị cho người dùng:

 Auto-Fix (Confidence: 94%) – Thay đổi đơn giản, an toàn cao

 Review Suggested (Confidence: 61%) – Breaking change phức tạp

 Manual Only (Confidence: 32%) – Đã từng rollback pattern này

4. Các Tính năng Cốt lõi

4.1 Multi-Ecosystem Support — Hỗ trợ đa hệ sinh thái

SUPPORTED ECOSYSTEMS		
JavaScript	Python	Java / Kotlin
package.json	requirements	pom.xml
yarn.lock	.txt / .toml	build.gradle
npm / yarn	Pipfile.lock	Maven Central
pnpm / bun	poetry.lock	Gradle Plugins
Go	Rust	.NET / C#
go.mod	Cargo.toml	.csproj
go.sum	Cargo.lock	NuGet packages
pkg.go.dev	crates.io	packages.config

4.2 Dependency Health Dashboard

PROJECT: my-saas-app

Health: 62/100



Package	Current	Latest	Status	Action
django	3.2.18	5.0.6	 EOL	[Auto-Fix]
numpy	1.21.0	1.26.4	 CVE	[Auto-Fix]
requests	2.28.0	2.32.2	 CVE	[Auto-Fix]
pandas	1.3.5	2.2.2	 Major	[Review]
sqlalchemy	1.4.46	2.0.30	 Major	[Review]
pytest	7.1.0	8.2.0	 Minor	[Auto-Fix]
black	22.3.0	24.4.0	 Minor	[Auto-Fix]
boto3	1.26.0	1.34.0	 Patch	[Auto-Fix]

 Fix All Critical

 Full Report

 Config

4.3 Smart Diff Preview — Xem trước trước khi áp dụng

Người dùng có thể xem toàn bộ thay đổi Agent định làm TRƯỚC khi nhấn Apply:

diff

 api/data_loader.py

Line 45:

```
- mask = np.bool(condition)
+ mask = np.bool_(condition) # np.bool removed in numpy 1.24
```

Line 89:

```
- arr = np.array(data, dtype=np.bool)
+ arr = np.array(data, dtype=np.bool_)
```

 analytics/report.py

Line 102:

```
- df = df.append(new_row, ignore_index=True)
+ df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
  # DataFrame.append() removed in pandas 2.0
```

Line 118:

```
- result = result.append(df2)
+ result = pd.concat([result, df2], ignore_index=True)
```

 requirements.txt

```
- numpy==1.21.0
+ numpy==1.26.4
- pandas==1.3.5
+ pandas==2.2.2
```

4.4 Changelogs Intelligence — Tóm tắt changelog thông minh

Thay vì đọc changelog dài hàng trăm trang, Agent tóm tắt CHỈ những gì liên quan:

 CHANGELOG SUMMARY: React 17 → 19 (chỉ những gì ảnh hưởng bạn)

 Breaking changes ảnh hưởng project của bạn:

1. ReactDOM.render() bị xóa → dùng createRoot()

Ảnh hưởng: src/index.js, src/main.jsx

Cách sửa: [Xem diff tự động]

2. defaultProps trên function components không còn hoạt động

Ảnh hưởng: components/UserCard.jsx

Cách sửa: Dùng default parameter values thay thế

 Breaking changes KHÔNG ảnh hưởng project của bạn:

- Legacy Context API (bạn dùng useContext)

- String refs (bạn dùng useRef)

- findDOMNode() (bạn không dùng)

[+ 9 breaking changes khác không liên quan]

 New features bạn có thể dùng được ngay:

- React Compiler (tự động memoization)

- use() hook cho async data

- Server Components improvements

4.5 Dependency Graph Visualizer — Bản đồ phụ thuộc

Trực quan hóa toàn bộ dependency tree:

- Node màu đỏ: CVE / EOL

- Node màu vàng: Major lag

- Edge màu đỏ: Conflict giữa 2 package

- Click vào node: Xem chi tiết, history, action

my-app



4.6 PR Auto-Generation — Tạo Pull Request sẵn sàng merge

Với dự án dùng Git, sau khi Auto-Fix:

Branch: depguard/fix-numpy-cve-2025-xxxx

PR Title:  [Security] Fix CVE-2025-XXXX: bump numpy 1.21 → 1.26

PR Description (auto-generated):

 Security Fix

****Severity:**** CRITICAL (CVSS 9.8)

****CVE:**** CVE-2025-XXXX – Remote Code Execution in numpy

Changes

- Updated numpy from 1.21.0 to 1.26.4
- Migrated deprecated APIs in 3 files (5 lines changed)

Code Changes

File	Change
api/data_loader.py	np.bool → np.bool_ (2 occurrences)
utils/matrix.py	np.complex → np.complex_ (1 occurrence)

Verification

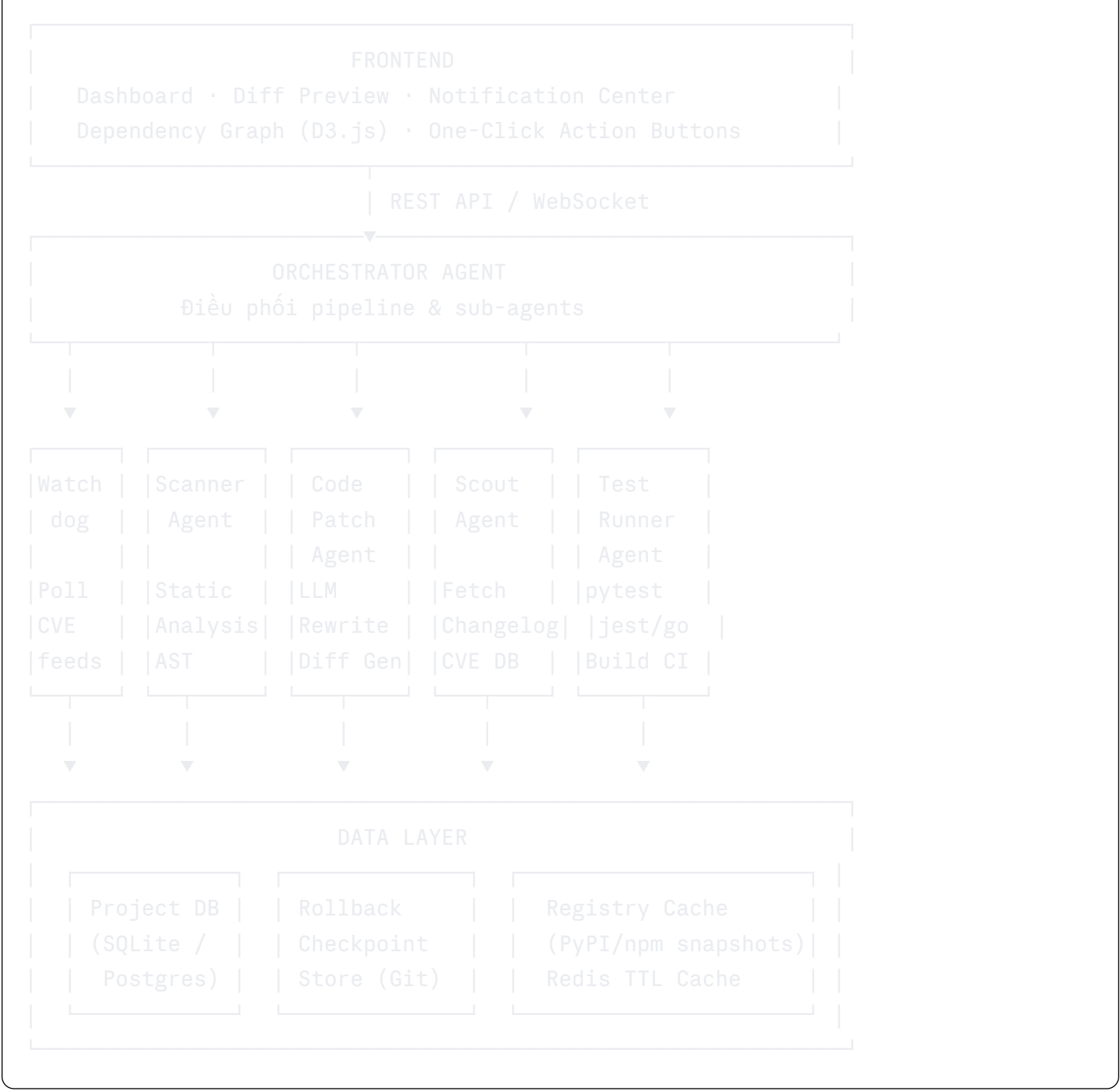
- ✓ All 47 unit tests pass
- ✓ All 12 integration tests pass
- ✓ Type checking (mypy) passed

Rollback

If issues arise: `depguard rollback ckpt\_20250601\_143022`

5. Kiến trúc Kỹ thuật

5.1 Sơ đồ Kiến trúc Tổng thể



5.2 Scanner Agent — AST-based Code Analysis

python

```
# Không dùng regex đơn giản – dùng AST để phân tích chính xác
import ast

class DeprecatedAPIScanner(ast.NodeVisitor):
    """
    Quét toàn bộ codebase tìm deprecated API usage
    Chính xác hơn regex: hiểu được context, scope, aliasing
    """
    DEPRECATED_NUMPY = {
        "np.bool": ("np.bool_", "1.20", "1.24"),
        "np.int": ("np.int_", "1.20", "1.24"),
        "np.float": ("np.float64", "1.20", "1.24"),
        "np.complex": ("np.complex_", "1.20", "1.24"),
    }

    def visit_Attribute(self, node):
        # Phát hiện: import numpy as np → np.bool
        if isinstance(node.value, ast.Name):
            full_name = f"{node.value.id}.{node.attr}"
            if full_name in self.DEPRECATED_NUMPY:
                new_api, deprecated_since, removed_in = \
                    self.DEPRECATED_NUMPY[full_name]
                self.report_issue(
                    location=node,
                    old_api=full_name,
                    new_api=new_api,
                    removed_in=removed_in
                )
            self.generic_visit(node)
```

5.3 Code Patch Agent — LLM-powered Rewrite

python

```
async def generate_code_patch(
    file_content: str,
    issue: DeprecationIssue,
    target_version: str
) -> CodePatch:
    """
    Dùng LLM để sinh code patch chính xác theo context
    Không chỉ tìm-thay đơn giản – hiểu ngữ nghĩa của đoạn code
    """

    prompt = f"""
    Bạn là expert về {issue.package} migration.

    File hiện tại (đoạn liên quan):
    ```python
 {file_content[issue.start:issue.end + 200]}
    ```

    Vấn đề: {issue.old_api} đã bị xóa trong {issue.package} {target_version}
    Thay thế bằng: {issue.new_api}

    Yêu cầu:
    1. Sửa đúng và đủ – không thay đổi logic
    2. Giữ nguyên style (indent, quotes, naming)
    3. Thêm comment ngắn giải thích lý do thay đổi
    4. Nếu migration phức tạp hơn rename đơn giản, giải thích rõ

    Trả về JSON: [{"patched_code": "...", "explanation": "..."}]
    """

    response = await llm.generate(prompt)
    return CodePatch.from_json(response)
```

5.4 Dependency Resolution Engine

python

```
class DependencyGraph:
    """
    Xây dựng và phân tích đồ thị phụ thuộc
    Tính toán thứ tự update an toàn (topological sort)
    """
    def compute_safe_update_order(
        self,
        packages_to_update: list[Package]
    ) -> list[UpdateStep]:

        # Xây dựng dependency graph
        graph = nx.DiGraph()
        for pkg in packages_to_update:
            for dep in pkg.dependencies:
                graph.add_edge(pkg.name, dep.name)

        # Topological sort → thứ tự update không conflict
        update_order = list(nx.topological_sort(graph))

        # Với mỗi bước, xác định: cần sửa code gì TRƯỚC
        steps = []
        for pkg_name in update_order:
            code_changes = self.scanner.find_breaking_usages(pkg_name)
            steps.append(UpdateStep(
                package=pkg_name,
                code_changes_first=code_changes,    # Sửa code trước
                then_update_version=True,           # Rồi mới update version
                then_run_tests=True,                # Cuối cùng chạy test
            ))

        return steps
```

6. Điểm Khác biệt & Sáng tạo

So sánh với các giải pháp hiện có:

Tiêu chí	Dependabot	Renovate	npm audit	DepGuard AI
Phát hiện version cũ	✓	✓	✓	✓
Phát hiện CVE	✓	✓	✓	✓
Đọc & hiểu code	✗	✗	✗	✓
Sửa code tự động	✗	✗	✗	✓
Tính thứ tự update	✗	Một phần	✗	✓
Tóm tắt changelog	✗	✗	✗	✓ (chi phần liên quan)
Rollback thông minh	Manual	Manual	Manual	✓ Auto
Học từ lịch sử	✗	✗	✗	✓
Đa ngôn ngữ	✓	✓	JS only	✓

Hai "Điểm Wow" Kỹ thuật:

1. AST-based Code Intelligence + LLM Patching

- Kết hợp phân tích tĩnh chính xác (AST) với khả năng hiểu ngữ nghĩa của LLM
- Không bao giờ tạo ra patch sai cú pháp, không bao giờ thay nhầm context

2. Dependency-Aware Update Sequencing

- Dùng topological sort trên dependency graph để tính thứ tự update
- Agent sửa code trước, update version sau — đúng thứ tự, không break build

7. Luồng Hoạt động Demo

Scenario: Một dự án Python bị lỗi thời và có CVE

[Bước 1] Kết nối project

- Developer kết nối GitHub repo hoặc upload project folder
- Scanner Agent phân tích: package.json, requirements.txt
- Health Score: 62/100 ⚠️
- Phát hiện: 3 CVE critical, 7 major lag, 12 minor lag

[Bước 2] Watchdog phát hiện CVE mới (real-time)

- Push notification: "CVE-2025-XXXX trong requests 2.28"
- Agent đã chuẩn bị sẵn Migration Plan + Code Diff
- Developer nhận thông báo khi đang ngủ, sáng dậy thấy

[Bước 3] Developer mở Dashboard, xem Smart Diff Preview

- 4 files cần thay đổi, 8 dòng code
- Xem trước từng thay đổi, hiểu rõ tại sao
- Confidence Score: 96% (pattern đơn giản, an toàn cao)

[Bước 4] Nhấn "🤖 Auto-Fix & Update"

- Checkpoint được tạo tự động
- Code Patch Agent sửa 8 dòng trong 4 file
- requirements.txt cập nhật
- pytest chạy: 47/47 tests pass ✅
- PR tự động tạo trên GitHub

[Bước 5] Xử lý Major Update (pandas 1.3 → 2.0)

- Confidence Score: 58% – "Suggest Review"
- Agent hiển thị: 12 breaking changes, chỉ 2 ảnh hưởng project
- Developer review, nhấn "Apply" từng thay đổi
- Sau mỗi apply: Agent chạy test ngay

[Bước 6] Nhìn lại sau 30 phút

- Health Score: 91/100 ✅
- 3 CVE đã vá, 8 packages đã update
- Rollback Memory ghi lại toàn bộ history

8. Tech Stack Đề xuất

Core AI/ML:

- **LLM:** Claude 3.5 Sonnet / GPT-4o (Code patching, changelog summarization)
- **Embeddings:** text-embedding-3-small (semantic search trong changelog)
- **Framework:** LangGraph (multi-agent orchestration)

Static Analysis:

- **Python:** `ast` module + `libcst` (preserves formatting)
- **JavaScript/TypeScript:** `@babel/parser` + `jscodeshift`
- **Java:** JavaParser / Spoon
- **Go:** `go/ast` package

Data Sources:

- **CVE Feeds:** NVD API, OSV.dev, GitHub Advisory Database
- **Package Registries:** PyPI JSON API, npm Registry API, Maven Central
- **Changelogs:** GitHub Releases API, PyPI changelog endpoint

Databases:

- **Project State:** PostgreSQL (projects, packages, history)
- **Registry Cache:** Redis (TTL-based caching, 6h refresh)
- **Rollback Store:** Git objects + SQLite metadata

Infrastructure:

- **Backend:** FastAPI (Python) + async workers (Celery)
 - **Frontend:** Next.js + TypeScript + shadcn/ui
 - **Visualization:** D3.js / Cytoscape.js (dependency graph)
 - **CI Integration:** GitHub Actions webhook / GitLab CI API
 - **Containerization:** Docker Compose
-

Kết luận

DepGuard AI không chỉ là một công cụ cảnh báo — nó là **người đồng đội kỹ thuật tự trị** với đầy đủ 4 trụ cột Agentic AI:

Trụ cột	Biểu hiện trong hệ thống
Goal-Driven	Health Score 0→100, 3 chế độ mục tiêu, Confidence-based action
Planning	Impact Graph, Safe Update Order (topo sort), Rollback Planning
Autonomous	Watchdog 24/7, Code Intelligence, One-Click Auto-Patch, Scout Agent
Feedback Loop	Build Verifier, Rollback Memory, User Feedback, Confidence Calibration

"The best security vulnerability is the one that never made it to production. The best technical debt is the one that was paid before it compounded."

Tài liệu này được chuẩn bị cho cuộc thi Hackathon — Agentic AI System Track
Phiên bản: 1.0 | Ngày: 2025